

AI Multi-Paper Knowledge Synthesizer

🎓 Interview Pitch Cheatsheet & Technical Presentation Guide

1. THE 30-SECOND ELEVATOR PITCH

"Researchers spend endless hours manually reading dozens of academic papers just to answer specific technical questions or craft literature reviews. To solve this efficiency bottleneck, I created the AI Multi-Paper Knowledge Synthesizer—an end-to-end RAG platform.

It ingests manuscripts via the ArXiv API or PDF uploads, indexes them using Hybrid Search (BM25 + 1024-dim BAAI embeddings) in Qdrant Vector DB, and leverages Llama 3.3 70B / Nemotron 70B to produce multi-paper syntheses with verifiable inline citations."

2. 4-PHASE SYSTEM ARCHITECTURE PITCH

Phase 1: Multi-Source Ingestion & Smart Chunking

- **Data Sources:** Automated live fetching via `ArXiv XML API` & custom PDF buffer parsing (`pdf-parse` / `pypdf`).
- **Recursive Character Chunking:** Hierarchical splitting using `[" ", " ", ". ", " "]` .
- **Chunk Parameters:** **700 characters** with a **100-character overlap** to maintain context across boundaries.

Phase 2: Dense Storage & Hybrid Sparse-Dense Retrieval

- **Dense Embeddings:** 1024-dimensional floating point vectors generated via `BAAI/bge-large-en-v1.5` .
- **Qdrant Vector Database:** Stored in an HNSW Graph Index with Cosine Distance & rich metadata payloads.
- **Hybrid Search Fusion:** Combines Dense Cosine Similarity with Sparse BM25 Keyword Matching:
 $Score = 0.65 \times CosineSimilarity(q, d) + 0.35 \times BM25Score(q, d)$

Phase 3: Grounded Reasoning & Citation Generation

- **LLM Inference:** Primary engine on `Groq (Llama 3.3 70B)` (25s timeout) with fallback to `Nvidia Nemotron 70B` .
- **Verifiable Inline Citations:** Answers enforce citation tags `[C1]` , `[C2]` mapped directly back to source paper excerpts to eliminate hallucinations.

Phase 4: Synthesis & Visualization Frontend

- **Modern UI:** Built with `React 19`, `TypeScript`, `Vite 6`, `Tailwind CSS v4` .
- **Key Features:** Synthesis workspace, automated Literature Reviews, Methodological Matrix grid, and interactive `SVG Knowledge Graph networks` .

3. TERMINOLOGY UPGRADE MATRIX

Original Raw Concept	Senior Engineering Interview Terminology
Fetch papers through ArXiv API	Automated multi-source ingestion via ArXiv XML API
Chunked and converted to embedding	Recursive character chunking paired with BAAI/bge-large 1024-dim dense embeddings
Stored to quadrant	Indexed in Qdrant Vector DB with HNSW graph algorithms and metadata payloads
Keyword matching + similarity search	Hybrid Sparse-Dense Retrieval combining Cosine Vector Similarity and BM25 score fusion
Context + query to LLM → answer	Grounded RAG synthesis chain returning cited reports & JSON comparison matrices

4. LIKELY INTERVIEW Q&A (SENIOR LEVEL)

Q1: Why Hybrid Search (BM25 + Cosine) instead of pure Vector Search?

"Dense vector search excels at conceptual similarity but can fail on exact keyword matches, math notation, formulas, or acronyms. Hybrid search fuses BM25 term frequency with dense vector cosine distance, ensuring high recall for both broad ideas and exact terms."

Q2: How do you prevent LLM Hallucinations?

"We strictly constrain the system prompt to retrieved paper chunks only. Every claim requires explicit citation tags like [C1], [C2], which link directly to source paragraphs in the Citation Inspector drawer."

Q3: Why 700-char chunks with 100-char overlap?

"Academic paragraphs typically span 500-800 characters. 700 chars encapsulates a complete logical thought without diluting vector density. The 100-char overlap guarantees semantic continuity across chunk boundaries."

Q4: How do you handle LLM API latency or down-time?

"The backend features automatic model fallback (Groq Llama-3.3-70b → HF Nemotron-70B) plus a 0ms offline local RAG reasoning engine grounded in paper paragraphs."